

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: PROYEC-20 — Validador de Prompts y Tokens

1. CONTEXTO

El equipo SDET requiere una utilidad interna de baja fricción para estimar tokens y latencia de prompts antes de integrarlos al orquestador principal. El requerimiento impone explícitamente una restricción de arquitectura: máximo 2 archivos en total. No hay usuarios concurrentes masivos, no hay persistencia de datos, no hay autenticación, y el backend es una simulación matemática sin conexión a LLMs reales. El ciclo de vida esperado es de herramienta interna/local.

2. DECISIÓN ARQUITECTÓNICA

****Arquitectura elegida: Single-File Dual-Layer (Static Frontend + Minimal API Backend)****

- `index.html` !' Monolito de presentación: HTML + CSS (Tailwind CDN) + JS vanilla con fetch API.
- `main.py` !' Microservidor FastAPI con un único endpoint POST `/api/analyze`, CORS habilitado para localhost, lógica de estimación inline sin capas adicionales.

****Stack tecnológico:****

- Frontend: HTML5 + Tailwind CSS (CDN) + JavaScript nativo (sin frameworks, sin bundlers, sin node_modules).
- Backend: Python 3.9+ + FastAPI + Uvicorn + asyncio (stdlib). Sin base de datos, sin caché, sin cola de mensajes.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS

- ****¿Por qué NO microservicios?**** Un solo desarrollador, un solo dominio funcional (análisis de texto), despliegue local. Microservicios añadirían overhead de red, orquestación y CI/CD sin ningún beneficio medible.
- ****¿Por qué NO React/Vue/Angular?**** La UI tiene exactamente 1 vista, 1 textarea, 1 botón y 3 tarjetas de resultado. Un framework SPA añadiría ~200KB de bundle, un proceso de build y complejidad de estado innecesaria. Fetch nativo resuelve el 100% del caso de uso.
- ****¿Por qué NO MongoDB/Redis/PostgreSQL?**** El sistema no persiste ningún dato. Agregar cualquier base de datos violaría YAGNI de forma flagrante y añadiría una dependencia de infraestructura sin retorno de valor.
- ****¿Por qué NO RabbitMQ/Celery para la latencia simulada?**** `asyncio.sleep()` es no-bloqueante, está en la stdlib de Python y resuelve exactamente el requerimiento de simulación de latencia. Una cola de mensajes sería ingeniería especulativa.
- ****¿Por qué NO una capa de repositorio o servicios separados en FastAPI?**** El endpoint realiza exactamente 2 operaciones: calcular `len(words) \* 1.3` y ejecutar `asyncio.sleep(random)`. Abstraer esto en capas de servicio/repositorio sería un anti-patrón de sobre-ingeniería para 8 líneas de lógica de negocio.
- ****¿Por qué Tailwind vía CDN y no instalado?**** El requerimiento lo especifica explícitamente. Además, elimina cualquier paso de build, manteniendo la herramienta ejecutable con doble-clic en el navegador.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- ****Riesgo 1 (Bajo):**** Si en el futuro se requiere persistir histórico de análisis, se deberá refactorizar `main.py` para agregar SQLite o PostgreSQL. Costo estimado: 2-4 horas de trabajo.
- ****Riesgo 2 (Bajo):**** Si se requiere autenticación, el CORS abierto a localhost deberá restringirse. Costo estimado: 1 hora.
- ****Riesgo 3 (Muy Bajo):**** Tailwind CDN requiere conexión a internet para cargar estilos. Para uso offline, se deberá descargar el CSS. Mitigación: documentar en README.
- ****Riesgo 4 (Aceptado):**** La estimación de tokens (`palabras \* 1.3`) es una aproximación matemática, no un tokenizador real (BPE/WordPiece). El requerimiento lo acepta explícitamente como simulación.

5. CONTRATO DE INTERFACES

- **Puerto backend:** `8000` (Uvicorn default)
- **Endpoint:** `POST http://localhost:8000/api/analyze`
- **Payload request:** `{ "prompt": "<string>" }`
- **Payload response:** `{ "estimated\_tokens": <int>, "simulated\_latency\_ms": <int>, "status": "Óptimo" | "Alerta" }`
- **Regla de negocio tokens:** `int(len(prompt.split()) \* 1.3)`
- **Regla de negocio latencia:** `random.randint(500, 2000)` ms via `asyncio.sleep(latency/1000)`
- **Regla de negocio status:** `"Alerta"` si `estimated\_tokens > 1000`, else `"Óptimo"`
- **CORS origins permitidos:** `["http://localhost", "http://127.0.0.1", "null"]` (null para file:// protocol)
- **Dependencias Python:** `fastapi`, `uvicorn[standard]` (sin extras innecesarios)

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 24,004

Tokens de salida (output): 7,737

Total tokens: 31,741

Horas-hombre estimadas: ~2.7 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,147 input | !'1,540 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"5,100 input | !'2,931 output | 2 llamadas

FRONTEND-DESARROLLO: !"7,124 input | !'2,471 output | 2 llamadas

QA-VALIDACION: !"9,633 input | !'795 output | 2 llamadas